JavaScript Array Method - push()

The `push()` method is one of the most commonly used array methods in JavaScript. It is used to add one or more elements to the **end** of an array.

Table of Contents

- What is push()?
- Why Use push()?
- Syntax
- Parameters
- Return Value
- How push() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is push()?

Definition

The `push()` method adds one or more elements to the **end** of an array.

It modifies the **original array** and returns the **new length** of the array.

Why Use push()?

We use `push()` when we need to:

- Add a new item to an array
- Store user input

- Add products to a shopping cart
- Save messages in a chat application
- Store API data
- Maintain dynamic lists

Syntax

```
array.push(element1);

array.push(element1, element2, element3);
```

Parameters

Parameter	Description
element1	First element to add
element2	Second element (Optional)
element3	Third element (Optional)

You can add **one or many elements** in a single call.

Return Value

`push()` returns the **new length** of the array.

Example

```
const fruits = ["Apple", "Mango"];

let length = fruits.push("Orange");

console.log(length);
```

Output

```
3
```

How push() Works

Before

```
["Apple", "Mango"]
```

↓

```
push("Orange")
```

↓

After

```
["Apple", "Mango", "Orange"]
```

The new element is always added to the **end** of the array.

Example 1 (Basic)

```
const fruits = [
    "Apple",
    "Mango"
];
fruits.push("Orange");
```

```
console.log(fruits);
```

Output

```
["Apple","Mango","Orange"]
```

Explanation

- Original array contains two elements.
- `push("Orange")` adds `"Orange"` at the end.
- The original array is modified.

Example 2 (Numbers)

```
const numbers = [
    10,
    20,
    30
];
numbers.push(40);
console.log(numbers);
```

Output

```
[10,20,30,40]
```

Example 3 (Multiple Elements)

```
const colors = [
    "Red",
    "Blue"
];
colors.push(
    "Green",
    "Black",
    "White"
);
console.log(colors);
```

Output

```
["Red","Blue","Green","Black","White"]
```

Example 4 (Store Return Value)

```
const fruits = [
    "Apple",
    "Mango"
];
let newLength = fruits.push("Orange");
```

```
console.log(newLength);

console.log(fruits);
```

Output

```
3

["Apple","Mango","Orange"]
```

Example 5 (Objects)

```
const users = [

    {

        name: "John"

    }

];

users.push({

    name: "Sara"

});

console.log(users);
```

Output

```
[
    { name: "John" },
    { name: "Sara" }
]
```

Example 6 (Booleans)

```
const values = [

    true,

    false

];

values.push(true);

console.log(values);
```

Output

```
[true,false,true]
```

Example 7 (Mixed Data Types)

```
const data = [

    "Rokon",

    23

];

data.push(
```

```
    true,
    null
);
console.log(data);
```

Output

```
["Rokon",23,true,null]
```

Example 8 (Empty Array)

```
const numbers = [];
numbers.push(100);
console.log(numbers);
```

Output

```
[100]
```

Real Project Example 1

Shopping Cart

```
const cart = [
    "Laptop",
```

```
    "Mouse"
];
cart.push("Keyboard");
console.log(cart);
```

Output

```
["Laptop","Mouse","Keyboard"]
```

Real Project Example 2

Student List

```
const students = [
    "Rahim",
    "Karim"
];
students.push("Rokon");
console.log(students);
```

Output

```
["Rahim","Karim","Rokon"]
```

Real Project Example 3

Notifications

```
const notifications = [];

notifications.push("New Message");

notifications.push("Payment Successful");

console.log(notifications);
```

Output

```
[
    "New Message",
    "Payment Successful"
]
```

Real Project Example 4

Chat Application

```
const messages = [];

messages.push("Hello");

messages.push("How are you?");

console.log(messages);
```

Output

```
[
    "Hello",
    "How are you?"
]
```

Real Project Example 5

Todo List

```
const todos = [

    "Study"

];

todos.push("Exercise");

todos.push("Read Book");

console.log(todos);
```

Output

```
[
    "Study",
    "Exercise",
    "Read Book"
]
```

Best Practices

Use `push()` to add elements at the end of an array.

```
cart.push(product);
```

Add multiple elements in one call when appropriate.

```
numbers.push(4, 5, 6);
```

Store the return value if you need the updated array length.

```
const length = numbers.push(100);
```

Use meaningful array names.

```
students

products

orders

messages
```

Common Mistakes

Mistake 1

Expecting `push()` to return the array.

Wrong

```
const result = numbers.push(4);

console.log(result);
```

Output

```
4
```

`push()` returns the **new length**, **not the array**.

Correct

```
numbers.push(4);

console.log(numbers);
```

Mistake 2

Using `push()` on a string.

Wrong

```
let name = "John";

name.push("A");
```

Output

```
TypeError
```

`push()` works only with arrays.

Mistake 3

Forgetting that `push()` changes the original array.

```
const numbers = [1,2];

numbers.push(3);
```

Now

```
numbers
```

becomes

```
[1,2,3]
```

Mistake 4

Using the wrong method.

To add at the **beginning**, use

```
unshift()
```

not

```
push()
```

Interview Questions

What does `push()` do?

It adds one or more elements to the **end** of an array.

Does `push()` modify the original array?

Yes.

What does `push()` return?

It returns the **new length** of the array.

Can `push()` add multiple elements?

Yes.

```
numbers.push(4,5,6);
```

Which end of the array does `push()` add elements to?

The **end** of the array.

Which method removes the last element?

```
pop()
```

Which method adds an element at the beginning?

```
unshift()
```

Summary Table

| Feature | Description | ||--------|-------------| | Method | `push()` | | Purpose | Add element(s) to the end of an array | | Parameters | One or more elements | | Returns | New array length | | Changes

Original Array | Yes | | Adds To | End of the array | | Common Uses | Shopping Cart, Todo List, Chat Messages, Notifications |

Quick Comparison

| Method | Action | Returns | Changes Original Array | ||-------|--------|---------|----------------------| | `push()` | Add to end | New length | Yes | | `pop()` | Remove from end | Removed element | Yes | | `unshift()` | Add to beginning | New length | Yes | | `shift()` | Remove from beginning | Removed element | Yes |

Final Notes

`push()` is one of the most frequently used array methods in JavaScript. It is widely used in:

- React state updates (when creating new arrays)
- Shopping carts
- Todo applications
- Chat systems
- Notifications
- API data processing
- Dynamic lists

Mastering `push()` is essential before learning other array methods like `pop()`, `shift()`, `unshift()`, `splice()`, and `concat()`.

JavaScript Array Method - `pop()`

The `pop()` method is one of the most commonly used array methods in JavaScript. It is used to remove the **last** element from an array.

Table of Contents

- What is pop()?
- Why Use pop()?
- Syntax
- Parameters
- Return Value

- How pop() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is pop()?

Definition

The `pop()` method removes the **last element** from an array.

It modifies the **original array** and returns the **removed element**.

Why Use pop()?

We use `pop()` when we need to:

- Remove the last item from an array
- Undo the last action
- Remove the latest notification
- Remove the last product from a shopping cart
- Remove the last message in a chat application
- Manage stack (LIFO) data structures

Syntax

```
array.pop();
```

Parameters

The `pop()` method **does not accept any parameters**.

Parameter	Description
None	No parameters required

Return Value

`pop()` returns the **removed element**.

Example

```
const fruits = ["Apple", "Mango", "Orange"];

let removedItem = fruits.pop();

console.log(removedItem);
```

Output

```
Orange
```

How pop() Works

Before

```
["Apple", "Mango", "Orange"]
```

↓

```
pop()
```

↓

After

```
["Apple", "Mango"]
```

The last element is always removed from the **end** of the array.

Basic Examples

Example 1 (Basic)

```
const fruits = [
    "Apple",
    "Mango",
    "Orange"
];
fruits.pop();
console.log(fruits);
```

Output

```
["Apple", "Mango"]
```

Explanation

- Original array contains three elements.
- `pop()` removes `"Orange"` from the end.
- The original array is modified.

Example 2 (Numbers)

```
const numbers = [
    10,
    20,
    30,
    40
];
numbers.pop();
console.log(numbers);
```

Output

```
[10, 20, 30]
```

Example 3 (Store Return Value)

```
const fruits = [
    "Apple",
    "Mango",
```

```
    "Orange"
];
let removedItem = fruits.pop();
console.log(removedItem);
console.log(fruits);
```

Output

```
Orange
["Apple", "Mango"]
```

Multiple Examples

Example 4 (Objects)

```
const users = [
    {
        name: "John"
    },
    {
        name: "Sara"
    }
];
const removedUser = users.pop();
console.log(removedUser);
```

```
console.log(users);
```

Output

```
{ name: "Sara" }

[
    { name: "John" }
]
```

Example 5 (Booleans)

```
const values = [

    true,

    false,

    true

];

values.pop();

console.log(values);
```

Output

```
[true, false]
```

Example 6 (Empty Array)

```
const arr = [];

const removed = arr.pop();

console.log(removed);

console.log(arr);
```

Output

```
undefined

[]
```

Example 7 (Stack Example)

```
const stack = [];

stack.push("HTML");

stack.push("CSS");

stack.push("JavaScript");

console.log(stack);

stack.pop();

console.log(stack);
```

Output

```
["HTML", "CSS", "JavaScript"]
```

```
["HTML", "CSS"]
```

Explanation

- `push()` adds elements to the end.
- `pop()` removes the most recently added element.
- Together they implement a **Stack (LIFO)**.

Real Project Examples

Example 1 (Shopping Cart)

```
const cart = [
    "Laptop",
    "Mouse",
    "Keyboard"
];
const removedProduct = cart.pop();
console.log("Removed:", removedProduct);
console.log(cart);
```

Output

```
Removed: Keyboard
["Laptop", "Mouse"]
```

Example 2 (Undo Feature)

```
const actions = [
    "Draw Circle",
    "Draw Square",
    "Draw Triangle"
];
const lastAction = actions.pop();
console.log("Undo:", lastAction);
console.log(actions);
```

Output

```
Undo: Draw Triangle
["Draw Circle", "Draw Square"]
```

Example 3 (Chat Messages)

```
const messages = [
    "Hello",
    "How are you?",
    "See you later"
];
const deletedMessage = messages.pop();
```

```
console.log(deletedMessage);

console.log(messages);
```

Output

```
See you later

["Hello", "How are you?"]
```

Best Practices

- Use `pop()` when removing the **last element**.
- Store the return value if you need the removed element later.
- Check if the array is empty before calling `pop()` .

```
if (array.length > 0) {

    array.pop();

}
```

Common Mistakes

Mistake 1

Expecting `pop()` to remove the first element.

Wrong

```
const numbers = [10, 20, 30];

numbers.pop();
```

Result

```
[10, 20]
```

Not

```
[20, 30]
```

Mistake 2

Passing an argument to `pop()` .

Wrong

```
const fruits = ["Apple", "Mango"];

fruits.pop("Apple");
```

Explanation

`pop()` ignores all arguments.

It always removes the **last element**.

Mistake 3

Using `pop()` on an empty array.

```
const arr = [];

console.log(arr.pop());
```

Output

```
undefined
```

Interview Questions

1. What does `pop()` do?

It removes the **last element** from an array.

2. Does `pop()` modify the original array?

Yes.

3. What does `pop()` return?

The removed element.

4. What happens if the array is empty?

It returns `undefined` .

5. Does `pop()` take any parameters?

No.

6. Which data structure commonly uses `push()` and `pop()` ?

A **Stack (LIFO)**.

Summary Table

Feature	Description
Method	`pop()`
Purpose	Remove the last element
Parameters	None
Return Value	Removed element
Modifies Original Array	Yes
Removes From	End
Empty Array Result	`undefined`
Time Complexity	**O(1)**

push() vs pop()

Method	Purpose	Position	Return Value
`push()`	Add element(s)	End	New array length
`pop()`	Remove element	End	Removed element

Key Points

- `pop()` removes the **last** element of an array.
- It **changes the original array**.
- It **returns the removed element**.
- If the array is empty, it returns `undefined` .
- `push()` and `pop()` are commonly used together to implement a **Stack (LIFO)**.